# Advanced Simulation Summative Coursework

**MSc in Statistics 2025/26, Imperial College London**

06051710

## Question 1: Filtering and smoothing for a nonlinear state space model

### 1a. Sampling from the SIR particle filter

For this problem, I chose as my importance density $q(x_n|y_n, x_{n-1})$ an approximation of the optimal proposal (that is $p(x_n|y_n, x_{n-1})$), by linearisations similar to the extended Kalman Filter (EKF). Although a bootstrap proposal would be easier, I felt the density $g(y_n|x_n)$ was relatively informative (or 'peaky'), which meant a more informative proposal seemed appropriate. Additionally, the notes suggest the EKF can give reasonable answers for models where the non-linearity is independent of the noise, which is the case for our model, as neither $V_n$ or $W_n$ appear in non-linear functions.

$X_n$ can be linearly approximated as $X_n = a_n + A_n X_{n-1} + B_n Z_n$ (where $Z_n \sim N(0,1)$) and $Y_n$ can be approximated as $Y_n = c_n + C_n X_n + W_n$ (where $W_n \sim N(0,1)$), both by a classic Taylor expansion. This leads to $B_n = \sqrt{0.1}$, $C_n = \frac{\psi_\theta(X_{n-1},0)}{10}$ and $D_n = 1$ (all other variables are not relevant for the proposal). The approximation of the optimal proposal[1] then takes the form $q(x_n|y_n, x_{n-1}) \propto N(m_n(x_{n-1}, y_n), S_n(x_{n-1}))$, where $m_n$ and $S_n$ are defined in Equation 1.

$$
\begin{aligned}
S_n^{-1}(x_{n-1}) &= (B_n B_n^T)^{-1} + C_n^T (D_n D_n^T)^{-1} C_n \\
S_n^{-1}(x_{n-1}) &= 10 + \frac{\psi_\theta(x_{n-1}, 0)^2}{100} \\
m_n(x_{n-1}, y_n) &= S_n(x_{n-1}) \Big( (B_n B_n^T)^{-1} \psi_\theta(x_{n-1}, 0) \\
&\qquad + C_n^T (D_n D_n^T)^{-1} (y_n - \phi_\theta(\psi_\theta(x_{n-1}, 0), 0) + C_n \psi_\theta(x_{n-1}, 0)) \Big) \\
m_n(x_{n-1}, y_n) &= S_n(x_{n-1}) \left( 10\psi_\theta(x_{n-1}, 0) + \frac{\psi_\theta(x_{n-1}, 0)}{10} \left( y_n + \frac{\psi_\theta(x_{n-1}, 0)^2}{20} \right) \right)
\end{aligned}
\tag{1}
$$

[1] this result is taken from the notes

For $q_\theta(x_0|y_0)$ the linearisation just leads to the bootstrap proposal, as linearising the likelihood of $y_0$ around the prior mean leads to $q_\theta(x_0|y_0) \sim N(0,1)$ (i.e. the bootstrap in this case).

Using multinomial resampling and the above proposals then led to the following particle approximations for $p(x_n|y_{0:n})$ in Figure 1 and Figure 2, where the densities are estimated via the default 'density' function in R, which uses a Gaussian kernel and rule-of-thumb bandwidth selector based on the standard deviation of the data. The red dotted line represents the true value of the hidden state, demonstrating how the SIR particle filter performed relatively accurately, even with N=50 (though increasing samples increased accuracy).

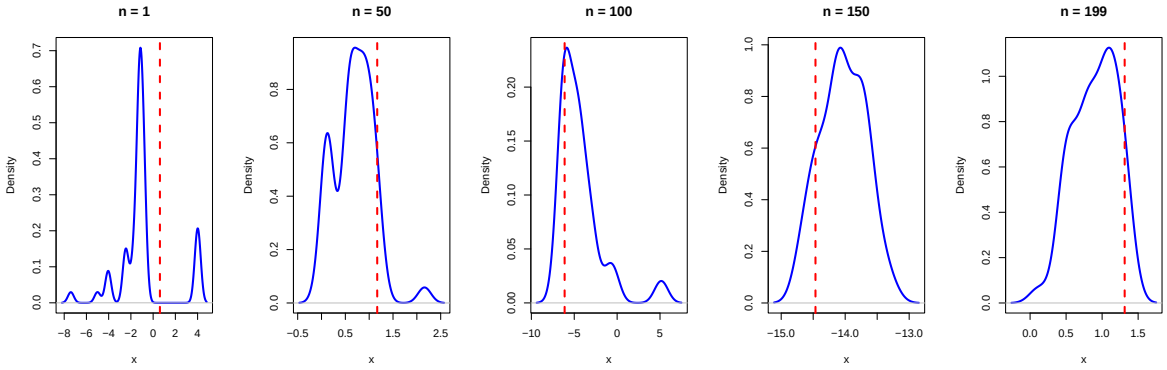


Figure 1: N=50 SIR particle filter approximations

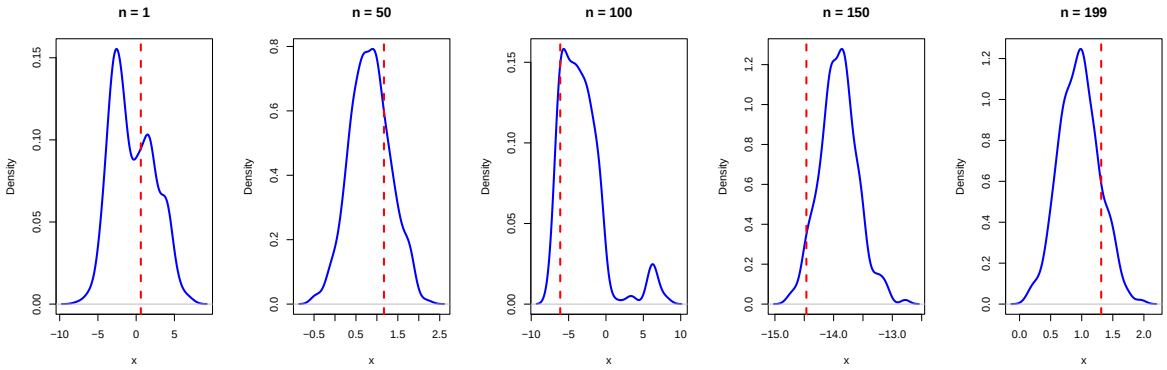


Figure 2: N=500 SIR particle filter approximations

### 1b. Sampling using a particle smoothing algorithm

For this question I chose to use a Forward Filtering Backwards Smoothing (FFBSm) algorithm, as I felt it was the best option I know of for particle smoothing problems (and did not require any hyperparameter tuning). It led to the filter approximations in Figure 3 and Figure 4 (via

the same smooth kernel density estimate as in 1a) which again seemed accurate in predicting hidden states. The terminology 'smoother' for $p(x_n|y_{0:T})$ seems justified, as visually speaking the density estimates appear 'smoother' than the SIR equivalents, likely due to using more information for their predictions.

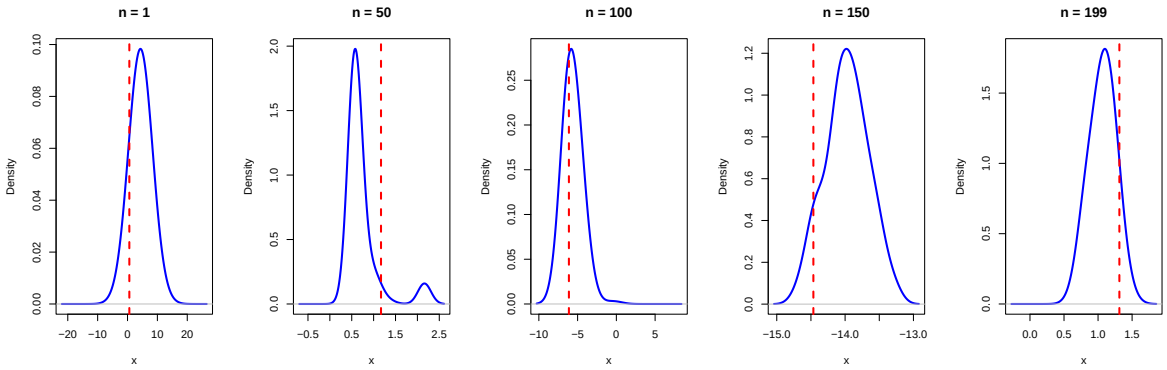


Figure 3: N=50 FFBSm Approximation

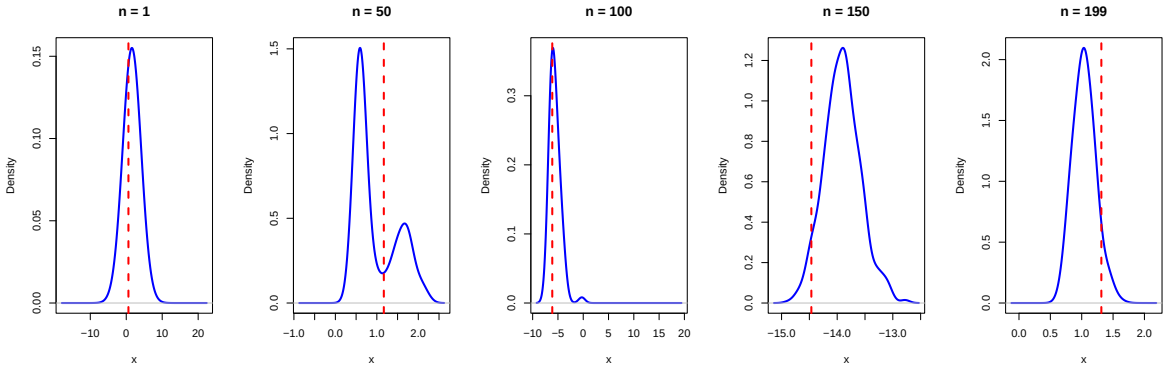


Figure 4: N=500 FFBSm Approximation

## 2a. Path Degeneracy for the SIR filter

The SIR filter has clear path degeneracy (for N=50). This is not clear if the paths of all particles are plotted due to the scale of the data, however, it is clearly demonstrated in Figure 5 where paths from n=190 onwards are plotted and compared to the paths from n=0 to 20. In fact, there is only a single path when going back from n=154[2], suggesting path degeneracy is a strong issue for the SIR filter.

[2]code for this is in the appendix

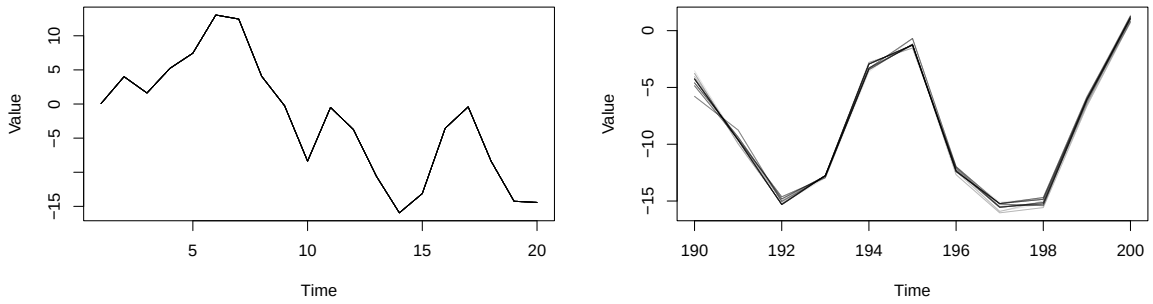


Figure 5: Visualising the start and end of the paths of the SIR filter

### 2b. Path Degeneracy for the FFBSm Algorithm

Although it is not possible to view 'paths' generated by the FFBSm Algorithm, as ultimately the forward run is identical to the generic SIR algorithm, we can still see the improvement in addressing path degeneracy when calculating particle approximations of $p(x_n|y_{0:T})$. For example, as explained previously, only one unique particle exists among all paths when calculating $p(x_n|y_{0:T})$ for $n \leq 154$ with the SIR filter, though in Figure 6 we can see there are almost 20 unique particles even for $n \leq 10$ (though this does vary depending on n).

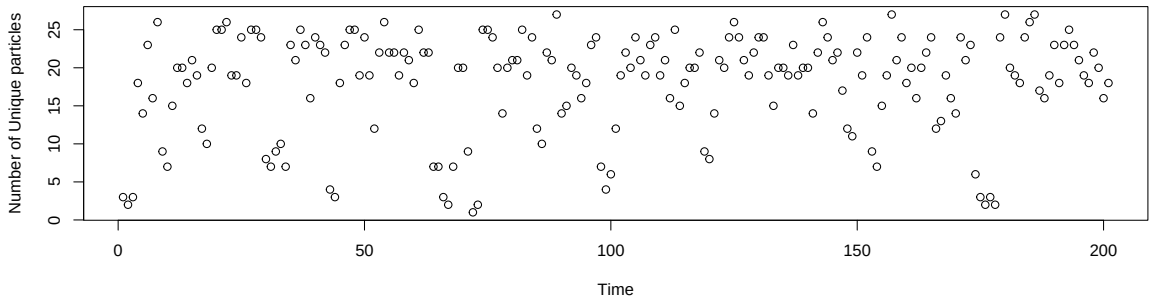


Figure 6: Visualising the number of unique particles in the FFBSm approximation

### 3. Particle Approximations for $E[X_n|y_{0:T}]$ and $E[X_n|y_{0:n}]$

To work out the SIR approximation of $E[X_n|y_{0:T}]$ I used the paths that I previously calculated to illustrate path degeneracy and calculated their mean. Figure 7 and Figure 8 display the true value of the hidden state at each time n in red, alongside both particle approximations

of $E[X_n|y_{0:T}]$ with the FFBSm approximation in green and the SIR approximation in black, as well as the filter approximation of $E[X_n|y_{0:n}]$ in blue. It is clear most approximations are very similar in value calculated, though the variance of the filter is much greater than both approaches used to calculate $p(x_n|y_{0:T})$.

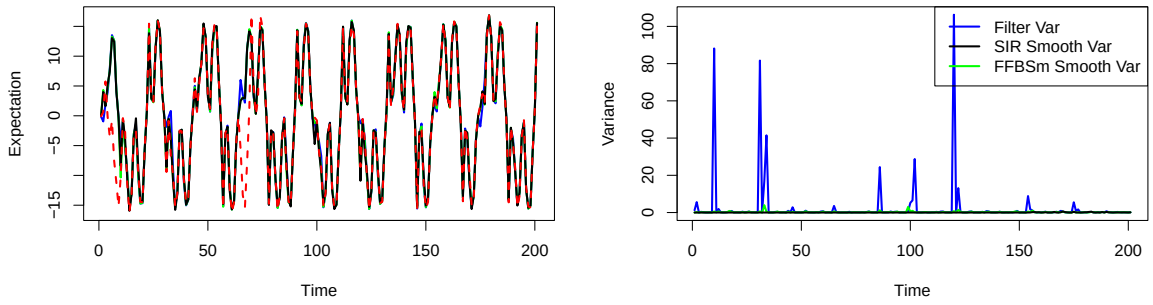


Figure 7: Filter vs Smoothing Estimates and Variance N=50

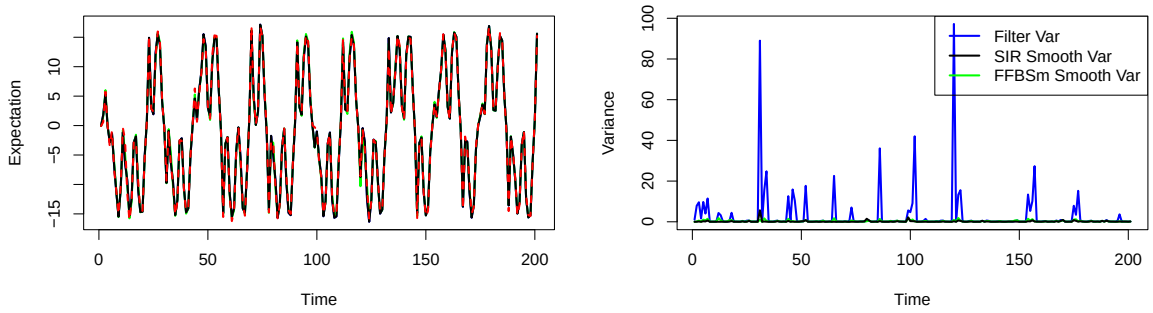


Figure 8: Filter vs Smoothing Estimates and Variance N=500

## 4. Comments on results

Visually speaking, when comparing the SIR approximation of $p(x_n|y_{0:n})$ to the FFBSm approximation of $p(x_n|y_{0:T})$ the FFBSm approach yields a much 'smoother' density, though both algorithms seem largely equal in ability to predict hidden states (the red lines always appear in regions with high densities). The 'smoothness' is also reflected in the increased variance of the filter compared to the smoother, seen in question 3, though again, it seems all algorithms are still able to calculate $E[X_n|y_{0:T}]$ to similar degrees of accuracy.

Smoothing uses more information than filtering and is offline, whereas filtering is online.

Though online algorithms are more useful when real-time parameter updates are sought after, having well-designed smoothing algorithms is necessary for parameter estimation.

Increasing N to 500 from 50 uniformly improved estimation accuracy in calculating the hidden state for all algorithms, most evidently demonstrated in part 3. I took T=200, though simulated 201 samples from the true model, as there was also an $x_0$ and $y_0$ term. The forward run of the SIR filter used multinomial resampling, as the basic version seemed to perform decently in terms of accuracy, though more advanced resampling techniques *could* have reduced the variance of estimates (they likely still would have had more variance than the smoothers).

The most computationally expensive algorithm by far was the FFBSm implementation with N=500, as it has a cost of $O(N^2T)$, leading to the largest runtime. On my device, the implementation of the forward run of the SIR algorithm (with N=500) took 0.65 seconds compared to 104 seconds for the implementation of the FFBSm algorithm.

## 5. Theoretical Properties for the smoothed additive functional

From the notes, our approximation of the smoothed additive functional for both parts a and b will be:

$$\hat{S}_n^\theta = \int \left[ \sum_{k=0}^{n} x_k \right] \hat{p}_\theta(dx_{0:n} \mid y_{0:n})$$

Where $\hat{p}_\theta(dx_{0:n}|y_{0:n}$ is approximated with the SIR filter using the paths created in part 2 to illustrate path degeneracy, compared to the smoothing approximation of it in part 1b.

Using just the SIR filter leads to the following theoretical bound for the variance:

$$\mathrm{Var}^N\left(\hat{S}_n^\theta\right) \geq D_\theta \frac{n^2}{N}$$

where $D_\theta$ is a constant that does not depend on n. Meanwhile the smoother will have theoretical bound:

$$\mathrm{Var}^N\left(\hat{S}_n^\theta\right) \geq H_\theta \frac{n}{N}$$

where $H_\theta$ is a constant that does not depend on n.

To numerically verify the above conditions I will run both a and b 50 times total, with N=50 and all conditions as before (I use N=50 as the focus is on the dependence on n, so this reduces computational cost). I will specifically run the filter independently for n=10,50,100,150,200 ten times each and then run the FFBSm algorithm over each run as well. Using these results, I will compute the smoothed additive functional and then compute the empirical variance over all

runs for each value of n. All of this can be recorded in a table like [MENTION REAL TABLE HERE]. Assessment of whether the theoretical bounds hold can be verified by calculating the proportion to which empirical variance rises between increases in n; for example, for the SIR approximation the empirical variance should rise by roughly 25 times for the increase from n=10 to n=50. I can then assess the difference between the expected theoretical rise in variance and the true rise intuitively to interpret whether the result holds.

### 6. Implementation of above procedure

## Code Appendix

```
knitr::opts_chunk$set(
  collapse = TRUE,
  comment = "#>"
)
include_solutions <- TRUE
set.seed(8)
require(rmarkdown)
require(knitr)
require(kableExtra)
# Put any library imports and other preamble here.
######
#Question 1a
######

#Simulating the dataset
x <- rep(0,201)
y <- rep(0,201)
x[1] <- rnorm(1)
y[1] <- (x[1]^2)/20 + rnorm(1)

for (i in 2:201){
  x[i] <- 0.5*x[i-1] + (25*x[i-1])/(1+(x[i-1])^2) +
    8*cos(1.2*(i-1))+rnorm(1, mean=0, sd=sqrt(0.1))
  y[i] <- (x[i]^2)/20 + rnorm(1)
}
#A single forward run of a SIR particle filter

#creates a matrix with N rows and n columns
#each column will have the Xi sampled at time i
```

```r
#weights will all be assumed to be 1/N, as the samples will be in-
#-the matrix only after resampling

psi0 <- function(x, n){
  return (0.5*x + (25*x)/(1+x^2) + 8*cos(1.2*n))
}

proposal <- function(x, y, n){
  psi <- psi0(x, n)
  Sn <- (10 + (psi^2)/100)^(-1)
  mn <- Sn * (10*psi + (psi/10) * (y + (psi^2)/20 ))
  return(list(mn = mn, Sn = Sn))
}

SIR <- function(N){
  samples <- matrix(rep(0,201*N), nrow=N, ncol=201)
  unweighted <- matrix(rep(0,201*N), nrow=N, ncol=201)
  sample_weights <- matrix(rep(0,201*N), nrow=N, ncol=201)
  ancestors <- matrix(0, nrow=N, ncol=201)

  ## Sampling first samples

  samples0 <- rep(0,N)
  weights0 <- rep(0,N)

  for (i in 1:N){
    samples0[i] <- rnorm(1, mean=0, sd=1)
    #compute unnormalised weights- target over proposal
    #in bootstrap case this is just g
    weights0[i] <- dnorm(y[1], mean=(samples0[i]^2)/20, sd=1)
  }

  # normalise weights
  weights0 <- weights0/sum(weights0)
  unweighted[,1] <- samples0
  sample_weights[,1]<-weights0

  #Resample (Multinomial Resampling)

  resamples <- sample(1:N, size = N, replace = TRUE, prob = weights0)
  ancestors[,1] <- resamples
  resamples0 <- rep(0,N)
```

```r
for (i in 1:N){
  resamples0[i] <- samples0[resamples[i]]
}

#update matrix
samples[,1] <- resamples0

## Sampling remaining 199*N samples
for (k in 2:201){
  samplesk <- rep(0,N)
  weightsk <- rep(0,N)


  for (i in 1:N){
    Xkprev <- samples[,k-1][i]
    prop <- proposal(x=Xkprev, y=y[k], n=k-1)
    samplesk[i] <- rnorm(1, mean=prop$mn, sd=sqrt(prop$Sn))

    fx <- psi0(Xkprev, k-1)
    transition <- dnorm(samplesk[i], mean=fx, sd=sqrt(0.1))
    likelihood <- dnorm(y[k], mean=(samplesk[i]^2)/20, sd=1)
    proposal_density <- dnorm(samplesk[i], mean=prop$mn, sd=sqrt(prop$Sn))




    log_w <- log(transition) + log(likelihood) - log(proposal_density)
    weightsk[i] <- exp(log_w)
  }
  #normalise weights
  weightsk <- weightsk/sum(weightsk)
  unweighted[,k] <- samplesk
  sample_weights[,k] <- weightsk

  #Multinomial Resampling
  resamples <- sample(1:N, size=N, replace=TRUE, prob=weightsk)
  ancestors[,k] <- resamples
  resamplesk <- rep(0,N)

  for (i in 1:N){
    resamplesk[i] <- samplesk[resamples[i]]
```

```
    }

    #update matrix
    samples[,k] <- resamplesk
  }
  return(list(samples = samples, ancestors = ancestors,
              unweighted=unweighted, sample_weights=sample_weights))
}

SIR50 <- SIR(50)
sir50 <- SIR50$samples
unweighted_sir50 <- SIR50$unweighted
weights_sir50 <- SIR50$sample_weights
SIR500 <- SIR(500)

#system.time({
#  SIR500 <- SIR(500)
#})- found the time is 0.649

sir500 <- SIR500$samples
unweighted_sir500 <- SIR500$unweighted
weights_sir500 <- SIR500$sample_weights
times = c(2,51,101,151,200)
par(mfrow = c(1, length(times)))

for (n in times) {
  d <- density(sir50[, n])
  plot(d, lwd = 2, col = "blue",
       main = paste("n =", n-1),
       xlab = "x", ylab = "Density")
  abline(v = x[n], col = "red", lwd = 2, lty = 2)
}

times = c(2,51,101,151,200)
par(mfrow = c(1, length(times)))

for (n in times) {
  d <- density(sir500[, n])
  plot(d, lwd = 2, col = "blue",
       main = paste("n =", n-1),
       xlab = "x", ylab = "Density")
  abline(v = x[n], col = "red", lwd = 2, lty = 2)
```

```r
}

######
#Question 1b
######

ffbsm <- function(samples, weights){
  N <- nrow(samples)
  T <- ncol(samples)

  back_weights <- matrix(0, nrow=N, ncol=T)
  back_weights[,T] <- weights[,T]

  for (k in (T-1):1){

    x_prev <- samples[,k]
    x_next <- samples[,k+1]

    # Transition matrix: [i, j] = f(x_next[j] | x_prev[i])
    trans_mat <- outer(
      x_prev, x_next,
      Vectorize(function(xp, xn) {
        dnorm(xn, mean = psi0(xp, k), sd = sqrt(0.1))
      })
    )

    # denom_j = sum_l weights[l,k] * f(x_next[j] | x_prev[l])
    denom <- as.vector(weights[,k] %*% trans_mat)

    # Avoid division by zero
    denom[denom == 0] <- 1e-12

    # Compute sum over j efficiently
    # back_weights[j,k+1] / denom[j]
    ratio <- back_weights[,k+1] / denom

    # sum_j ratio[j] * f(x_next[j] | x_prev[i])
    back_weights[,k] <- weights[,k] * (trans_mat %*% ratio)

    # normalise
    back_weights[,k] <- back_weights[,k] / sum(back_weights[,k])
  }
```

```
  return(back_weights)
}

system.time({
  ffbsm_weights1 <- ffbsm(unweighted_sir50, weights_sir50)
})

system.time({
  ffbsm_weights2 <- ffbsm(unweighted_sir500, weights_sir500)
})



times = c(2,51,101,151,200)
par(mfrow = c(1, length(times)))

for (n in times) {
  d <- density(unweighted_sir50[,n],
               weights = ffbsm_weights1[,n])
  plot(d, lwd = 2, col = "blue",
       main = paste("n =", n-1),
       xlab = "x", ylab = "Density")
  abline(v = x[n], col = "red", lwd = 2, lty = 2)
}


times = c(2,51,101,151,200)
par(mfrow = c(1, length(times)))

for (n in times) {
  d <- density(unweighted_sir500[,n],
               weights = ffbsm_weights2[,n])
  plot(d, lwd = 2, col = "blue",
       main = paste("n =", n-1),
       xlab = "x", ylab = "Density")
  abline(v = x[n], col = "red", lwd = 2, lty = 2)
}
######
#Question 2a
######

path_generator <- function(anc, samples){
```

```
  paths <- matrix(0, nrow=length(samples[,1]), ncol=201)
  paths[,201] <- samples[,201]
  indices <- 1:length(samples[,1])
  idx <- 201
  while (idx > 1) {
    indices <- anc[indices, idx]
    paths[, idx-1] <- samples[indices, idx-1]
    idx <- idx - 1
  }
  return(paths)
}

paths50 <- path_generator(SIR50$ancestors, SIR50$samples)
par(mfrow=c(1,2))

matplot(t(paths50[,0:20]), type = "l", lty = 1, col = rgb(0, 0, 0, 0.1),
        xlab = "Time", ylab = "Value")
matplot(190:200, t(paths50[,190:200]), type = "l", lty = 1,
        col = rgb(0, 0, 0, 0.1),
        xlab = "Time", ylab = "Value")
length(unique(paths50[,201]))
length(unique(paths50[,200]))
length(unique(paths50[,170]))
length(unique(paths50[,160]))
length(unique(paths50[,154]))
length(unique(paths50[,100]))
length(unique(paths50[,20]))
length(unique(paths50[,1]))

######
#Question 2b
######

unique_particles <- function(samples, weights){
  # samples: 50 x 201 matrix
  # weights: same size

  n_particles <- nrow(samples)
  n_steps <- ncol(samples)
  unique_counts <- numeric(n_steps)

  for (k in 1:n_steps){
```

```
    weightsk <- weights[, k]
    resamples <- sample(1:n_particles, size = n_particles,
                        replace = TRUE, prob = weightsk)
    resampled_values <- samples[resamples, k]
    unique_counts[k] <- length(unique(resampled_values))
  }
  return(unique_counts)
}

plot(1:201, unique_particles(sir50, ffbsm_weights1),
     xlab="Time",
     ylab="Number of Unique particles")

######
#Question 3 Plot 1
######

smooth_mean <- function(samples, back_weights){
  colSums(samples * back_weights)
}


EX_smooth <- colMeans(paths50)
EX_smooth_ffbsm <- smooth_mean(unweighted_sir50, ffbsm_weights1)
EX_filter <- colMeans(SIR50$samples)

par(mfrow = c(1, 2))

plot(1:201, EX_filter, type="l", lwd=2,
     xlab="Time", ylab="Expectation",
     ylim = range(c(EX_filter, EX_smooth, x)),
     col="blue")

lines(1:201, EX_smooth_ffbsm, col="green", lwd=2)
lines(1:201, EX_smooth, col="black", lwd=2)
lines(1:201, x, col="red", lty=2, lwd=2)

var_calc <- function(samples){
  apply(samples, 2, var)
}

VAR_filter_50 <- var_calc(sir50)
```

```
VAR_smooth_paths50 <- var_calc(paths50)

smooth_var <- function(samples, weights){
  means <- colSums(samples * weights)
  vars <- colSums(weights * (samples - matrix(means,
                    nrow=nrow(samples),
                    ncol=ncol(samples),
                    byrow=TRUE))^2)
  return(vars)
}

VAR_smooth_ffbsm_50 <- smooth_var(unweighted_sir50, ffbsm_weights1)

plot(1:201, VAR_filter_50, type="l", lwd=2,
     xlab="Time", ylab="Variance",
     ylim = range(c(VAR_filter_50,
                    VAR_smooth_paths50,
                    VAR_smooth_ffbsm_50)),
     col="blue")

lines(1:201, VAR_smooth_ffbsm_50, col="green", lwd=2)
lines(1:201, VAR_smooth_paths50, col="black", lwd=2)


legend("topright",
       legend=c("Filter Var", "SIR Smooth Var", "FFBSm Smooth Var"),
       col=c("blue","black","green"),
       lwd=2)

######
#Question 3 plot 2 (N=500)
######

paths500 <- path_generator(SIR500$ancestors, SIR500$samples)


EX_smooth500 <- colMeans(paths500)
EX_smooth_ffbsm500 <- smooth_mean(unweighted_sir500, ffbsm_weights2)
EX_filter500 <- colMeans(SIR500$samples)

par(mfrow = c(1, 2))
plot(1:201, EX_smooth500, type="l", lwd=2,
```

```
     xlab="Time", ylab="Expectation",
     ylim = range(c(EX_filter500, EX_smooth500, x)),
     col="blue")

lines(1:201, EX_smooth_ffbsm500, col="green", lwd=2)
lines(1:201, EX_smooth500, col="black", lwd=2)
lines(1:201, x, col="red", lty=2, lwd=2)

VAR_filter_500 <- var_calc(sir500)
VAR_smooth_paths500 <- var_calc(paths500)
VAR_smooth_ffbsm_500 <- smooth_var(unweighted_sir500, ffbsm_weights2)

plot(1:201, VAR_filter_500, type="l", lwd=2,
     xlab="Time", ylab="Variance",
     ylim = range(c(VAR_filter_500,
                    VAR_smooth_paths500,
                    VAR_smooth_ffbsm_500)),
     col="blue")

lines(1:201, VAR_smooth_ffbsm_500, col="green", lwd=2)
lines(1:201, VAR_smooth_paths500, col="black", lwd=2)


legend("topright",
       legend=c("Filter Var", "SIR Smooth Var", "FFBSm Smooth Var"),
       col=c("blue","black","green"),
       lwd=2)


###
#Question 1 Part 6
###

'
compute_S <- function(paths){
  # paths: N x T matrix
  apply(paths, 1, cumsum)  # cumulative sum per particle
}

compute_S_ffbsm <- function(samples, weights){
  N <- nrow(samples)
  T <- ncol(samples)
```

```
  S_vals <- matrix(0, nrow=N, ncol=T)

  for (i in 1:N){
    S_vals[i,] <- cumsum(samples[i,])
  }

  # weighted expectation at each time
  colSums(S_vals * weights)
}

n_vals <- c(10, 50, 100, 150, 200)
n_runs <- 10

sir_results <- matrix(0, nrow=n_runs, ncol=length(n_vals))
ffbsm_results <- matrix(0, nrow=n_runs, ncol=length(n_vals))

for (r in 1:n_runs){
  sim <- SIR(50)

  paths <- path_generator(sim$ancestors, sim$samples)
  S_paths <- compute_S(paths)

  weights_ffbsm <- ffbsm(sim$unweighted, sim$sample_weights)
  S_ffbsm <- compute_S_ffbsm(sim$unweighted, weights_ffbsm)

  for (j in 1:length(n_vals)){
    n <- n_vals[j]

    # SIR estimate (mean over particles)
    sir_results[r, j] <- mean(S_paths[, n])

    # FFBSm estimate
    ffbsm_results[r, j] <- S_ffbsm[n]
  }
}

sir_var <- apply(sir_results, 2, var)
ffbsm_var <- apply(ffbsm_results, 2, var)

data.frame(
  n = n_vals,
  SIR_variance = sir_var,
```

```
  FFBSm_variance = ffbsm_var
)

plot(n_vals, sir_var, type="b", lwd=2, col="blue",
     xlab="n", ylab="Variance")
lines(n_vals, ffbsm_var, type="b", col="red", lwd=2)
legend("topleft", legend=c("SIR", "FFBSm"),
       col=c("blue","red"), lwd=2)
'
```